CSE 309: Theory of Automata
Context-Free Languages

Shahid Hussain

shahidhussain@iba.edu.pk

February/March 2024

School of Mathemtics and Computer Science
Institue of Business Administration

Context-Free Grammars

- We know that there are languages which are not regular
- Or in other words, we can't have **finite automata** and **regular expressions** for them
- We will study a more powerful method of describing languages called **context-free grammars** (CFG)
- CFG were first used to study natural languages
- Most compilers use CFG to describe the syntax of the corresponding programming languages
- These compilers **parse** the program and extract the meaning from it
- The collection of languages associated with CFG is called **context-free languages** (CFL)
- We will see a corresponding machine to recognize CFL called **pushdown automata**

Context-Free Grammars

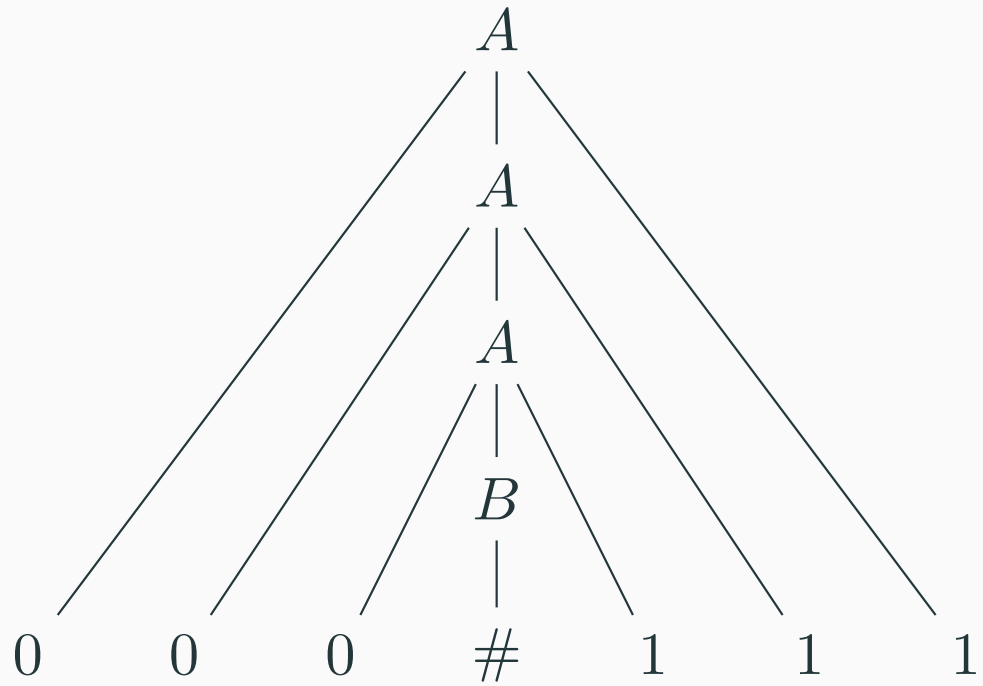
- Consider the following grammar G_1:

$$\begin{aligned} A &\rightarrow 0A1 \\ A &\rightarrow B \\ B &\rightarrow \# \end{aligned}$$

- This is collection of **substitution rules** or **productions**
- Capital letters represent **variables** while small letters represent **terminals**
- In G_1, A is the **start variable**
- 0, 1, and $\#$ are terminals (alphabet)
- G_1 generates the string $000\#111$

Context-Free Grammars: Derivation

- $A \Rightarrow 0A1 \Rightarrow 00A11 \Rightarrow 000B111 \Rightarrow 000\#111$
- Following is the parse tree for the corresponding derivation



- All strings generated this way make the **language of the grammar**
- $L(G_1) = \{0^n\#1^n : n \geq 0\}$

Context-Free Grammar

Context-Free Grammar

A **context-free grammar** is a 4-tuple (V, Σ, R, S), where

1. V is a finite set of **variables**
2. Σ is a finite set, disjoint from V, of **terminals**
3. R is a finite set of **rules**
4. $S \in V$ is the start variable.

- If u, v, w are strings of variables and terminals, and $A \to w$ is a rule of the grammar, we say uAv **yields** uwv, written $uAv \Rightarrow uwv$
- Or u **derives** v, written $u \overset{*}{\Rightarrow} v$
- $L(G)$ is $\{w \in \Sigma^* : S \overset{*}{\Rightarrow} w\}$

Examples

- $G = \{\{S\}, \{\mathtt{a}, \mathtt{b}\}, R, S\}$ and set of rules R

$$S \to \mathtt{a}\ S\ \mathtt{b} \mid SS \mid \epsilon.$$

Examples

- $G = \{\{S\}, \{\mathtt{a}, \mathtt{b}\}, R, S\}$ and set of rules R

$$S \to \mathtt{a}\ S\ \mathtt{b} \mid SS \mid \epsilon.$$

- $G = \{V, \Sigma, R, \langle \textsc{expr} \rangle\}$, the rules are

$$\begin{aligned} \langle \textsc{expr} \rangle &\to \langle \textsc{expr} \rangle + \langle \textsc{term} \rangle \mid \langle \textsc{term} \rangle \\ \langle \textsc{term} \rangle &\to \langle \textsc{term} \rangle \times \langle \textsc{factor} \rangle \mid \langle \textsc{factor} \rangle \\ \langle \textsc{factor} \rangle &\to (\langle \textsc{expr} \rangle) \mid \mathtt{a} \end{aligned}$$

Examples

- $G = \{\{S\}, \{\mathtt{a}, \mathtt{b}\}, R, S\}$ and set of rules R

$$S \to \mathtt{a}\ S\ \mathtt{b} \mid SS \mid \epsilon.$$

- $G = \{V, \Sigma, R, \langle \textsc{expr} \rangle\}$, the rules are

$$\begin{aligned} \langle \textsc{expr} \rangle &\to \langle \textsc{expr} \rangle + \langle \textsc{term} \rangle \mid \langle \textsc{term} \rangle \\ \langle \textsc{term} \rangle &\to \langle \textsc{term} \rangle \times \langle \textsc{factor} \rangle \mid \langle \textsc{factor} \rangle \\ \langle \textsc{factor} \rangle &\to (\langle \textsc{expr} \rangle) \mid \mathtt{a} \end{aligned}$$

- Now contrast with

$$\langle \textsc{expr} \rangle \to \langle \textsc{expr} \rangle + \langle \textsc{expr} \rangle \mid \langle \textsc{expr} \rangle \times \langle \textsc{expr} \rangle \mid (\langle \textsc{expr} \rangle) \mid \mathtt{a}$$

Designing CFG's

- There is no general algorithm, there are few tricks
- Union of two or more CFG's is a CFG
- Let $S_1, S_2, \ldots, S_k$ be two k different CFG's with S_i being the start variable for the CFG i
- We can merge these grammars into one by merging the rules for all and making the rule $S \to S_1 \mid S_2 \mid \cdots \mid S_k$

Designing CFG's

- There is no general algorithm, there are few tricks
- Union of two or more CFG's is a CFG
- Let $S_1, S_2, \ldots, S_k$ be two k different CFG's with S_i being the start variable for the CFG i
- We can merge these grammars into one by merging the rules for all and making the rule $S \to S_1 \mid S_2 \mid \cdots \mid S_k$
- Let $L = \{0^n 1^n : n \geq 0\} \cup \{1^n 0^n : n \geq 0\}$ then we have:

$$\begin{aligned} S_1 &\to 0S_1 1 \mid \epsilon \\ S_2 &\to 1S_2 0 \mid \epsilon \end{aligned}$$

respectively for the two subsets in L, we can construct CFG for L as

$$\begin{aligned} S &\to S_1 \mid S_2 \\ S_1 &\to 0S_1 1 \mid \epsilon \\ S_2 &\to 1S_2 0 \mid \epsilon \end{aligned}$$

Designing CFG's

- Constructing a CFG for a regular language is easy
- Let L be a regular language and M is the DFA for L
- Make a variable R_i for each state q_i in M
- Add rule $R_i \to aR_j$ to the CFG if $\delta(q_i, a) = q_j$
- Make R_0 the start variable of the grammar (q_0 is the initial state of M)

Designing CFG's

- Certain CFG's have substrings linked together such as

$$\{0^n 1^n : n \geq 0\}$$

- These cases can be handled by using a rule of the form

$$R \to uRv$$

Ambiguity

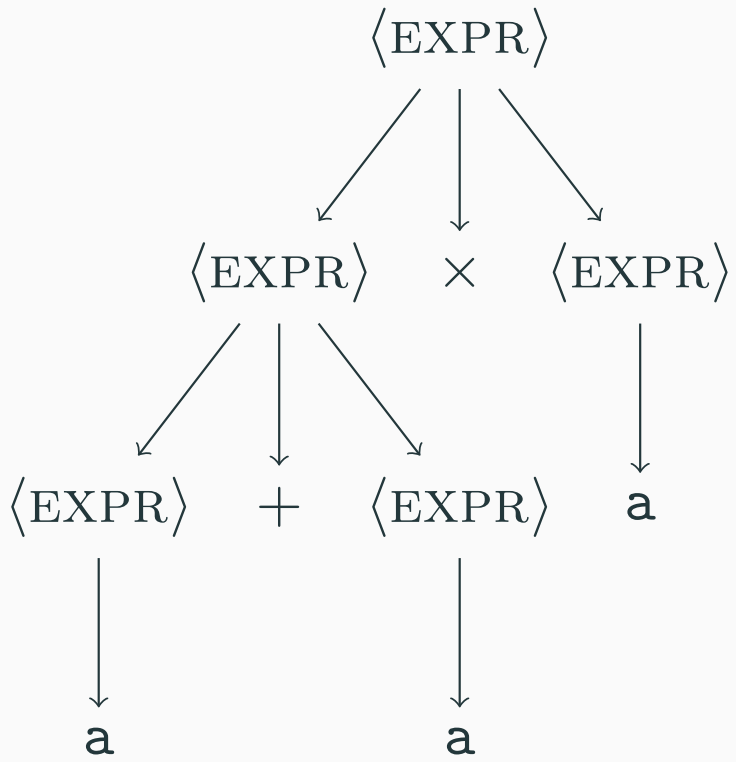
- If a grammar generates the same string in several different ways, we say that string is derived **ambiguously** in that grammar
- If a grammar generates some string ambiguously we say that the grammar is **ambiguous**, e.g.,

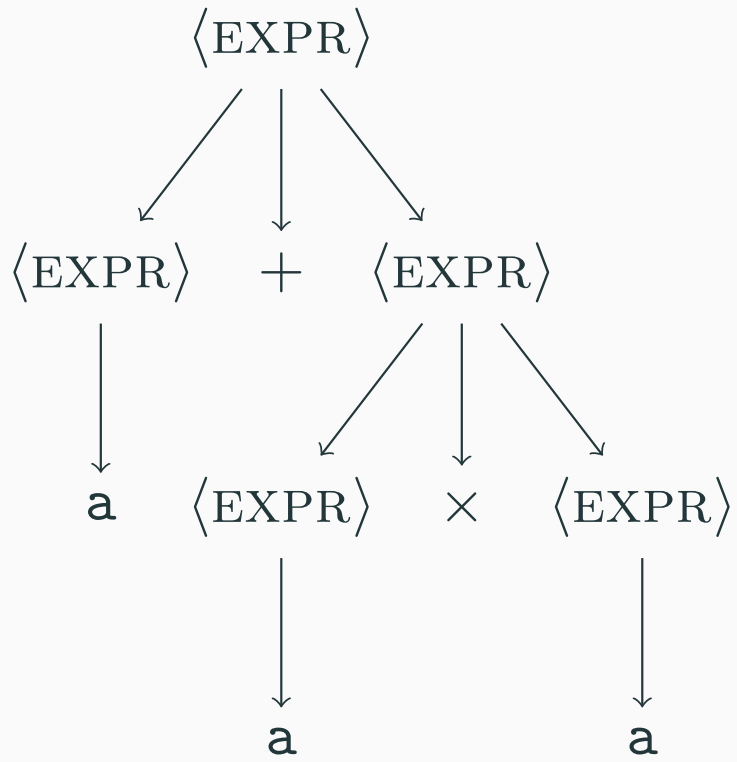
$$\langle \text{EXPR} \rangle \rightarrow \langle \text{EXPR} \rangle + \langle \text{EXPR} \rangle \mid \langle \text{EXPR} \rangle \times \langle \text{EXPR} \rangle \mid (\langle \text{EXPR} \rangle) \mid \mathtt{a}$$

- This grammar generates the string $\mathtt{a} + \mathtt{a} \times \mathtt{a}$ ambiguously

Ambiguity

- This grammar generates the string $a + a \times a$ ambiguously





Ambiguity

Ambiguity

A string is derived **ambiguously** in context-free grammar G if it has two or more different leftmost derivations. Grammar G is **ambiguous** if it generates some string ambiguously.

Ambiguity

Ambiguity

A string is derived **ambiguously** in context-free grammar G if it has two or more different leftmost derivations. Grammar G is **ambiguous** if it generates some string ambiguously.

- Sometimes it is possible to find an unambiguous alternative grammar that generates the same language
- Some languages can be generated only by an ambiguous grammars called **inherently ambiguous** e.g.,

$$\{a^i b^j c^k : i = j \vee j = k\}$$

Chomsky Normal Form

Chomsky Normal Form

A context-free grammar is in **Chomsky normal form** if every rule is of the form

$$\begin{aligned} A &\to BC \\ A &\to a \end{aligned}$$

where a is any terminal and A, B, C are any variables—except that B and C may not be the start variables. In addition, we permit the rule $S \to \epsilon$, where S is the start variable.

Chomsky Normal Form

Chomsky Normal Form

A context-free grammar is in **Chomsky normal form** if every rule is of the form

$$\begin{aligned} A &\to BC \\ A &\to a \end{aligned}$$

where a is any terminal and A, B, C are any variables—except that B and C may not be the start variables. In addition, we permit the rule $S \to \epsilon$, where S is the start variable.

Theorem 1

Any context-free language is generated by a context-free grammar in Chomksy normal form.

Proof for Theorem

- We add a new start variable S_0 and the rule $S_0 \to S$
- We take take care of ϵ rules
- We remove an ϵ-rule $A \to \epsilon$ where A is not the start variable
- Then for each occurrence of an A on the right-hand side of the rule, we add a new rule with the occurrence is deleted i.e., if $R \to uAv$ is a rule we add rule $R \to uv$
- If we have a rule $R \to A$ we add $R \to \epsilon$ unless we had previously removed the rule $R \to \epsilon$, we repeat till all ϵ rules eliminated
- We handle unit rule $A \to B$, i.e., whenever $B \to u$ appears we add the rule $A \to u$, unless it was already removed
- Finally, we convert all remaining rules into the proper form by replacing $A \to u_1 u_2 \cdots u_k$, $k \geq 3$ with the rules $A \to u_1 A_1$, $A_1 \to u_2 A_2$, ..., $A_{k-2} \to u_{k-1} u_k$
- If $k = 2$ we replace any terminal u_i in the preceding rule(s)

Chomsky Normal Form

- Convert following grammar G in Chomsky normal form

$$\begin{aligned} S &\rightarrow ASA \mid aB \\ A &\rightarrow B \mid S \\ B &\rightarrow b \mid \epsilon \end{aligned}$$

Chomsky Normal Form

- Convert following grammar G in Chomsky normal form

$$\begin{aligned} S &\rightarrow ASA \mid aB \\ A &\rightarrow B \mid S \\ B &\rightarrow b \mid \epsilon \end{aligned}$$

- We get

$$\begin{aligned} S_0 &\rightarrow AA_1 \mid UB \mid a \mid SA \mid AS \\ S &\rightarrow AA_1 \mid UB \mid a \mid SA \mid AS \\ A &\rightarrow b \mid AA_1 \mid UB \mid a \mid SA \mid AS \\ A_1 &\rightarrow SA \\ U &\rightarrow a \\ B &\rightarrow b \end{aligned}$$

Pushdown Automata (PDA)

- **Pushdown automata** are like nondeterministic finite automata with an extra component **stack**
- Computational power of PDA is equivalent to CFG
- We can get similar result like a language is context-free if and only if there is a PDA that recognizes it
- PDA can read and write symbols from/to the stack
- Interestingly, nondeterministic pushdown automata are **not** equivalent to deterministic pushdown automata in power
- That is NPDA can recognize certain languages which no DPDA can recognize

Pushdown Automata (PDA)

Pushdown Automata

A **pushdown automata** is a 6-tuple $(Q, \Sigma, \Gamma, \delta, q_0, F)$, where Q, Σ, Γ, and F are all finite sets, and

1. Q is the set of states,
2. Σ is the input alphabet,
3. Γ is the stack alphabet,
4. $\delta : Q \times \Sigma_\epsilon \to \mathcal{P}(Q \times \Gamma_\epsilon)$ is the transition function,
5. $q_0 \in Q$ is the start state, and
6. $F \subseteq Q$ is the set of accept states.

Computation by PDA

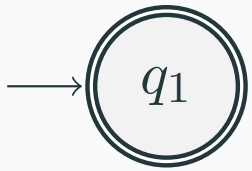
- A pushdown automata $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$ computes as follows:
- It accepts input w if w can be written as $w = w_1 w_2 \cdots w_m$ where each $w_i \in \Sigma_\epsilon$ and sequence of states $r_0, r_1, \ldots, r_m \in Q$ and strings $s_0, s_1, \ldots, s_m \in \Gamma^*$ exists that satisfy following: (the string s_i represents the sequence of stack contents that M has on the accepting branch of the computation)
  1. $r_0 = q_0$ and $s_0 = \epsilon$ (i.e., M starts with and empty stack)
  2. $0 \leq i \leq m - 1$, we have $(r_{i+1}, b) \in \delta(r_i, w_{i+1}, a)$, where $s_i = at$ and $s_{i+1} = b$ for some $a, b \in \Gamma_\epsilon$ and $t \in \Gamma^*$
  3. $r_m \in F$

Example PDA

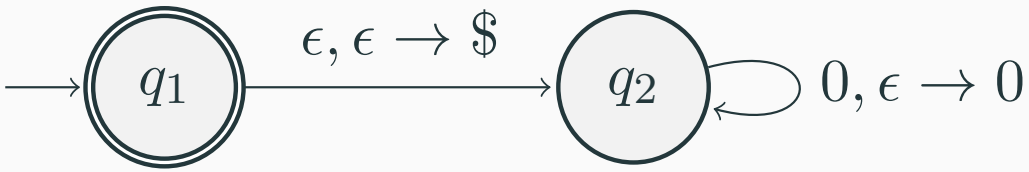
- Let $L = \{0^n 1^n : n \geq 0\}$, let $M = (Q, \Sigma, \Gamma, \delta, q_1, F)$, where
- $Q = \{q_1, q_2, q_3, q_4\}$, $\Sigma = \{0, 1\}$, $\Gamma = \{0, \$\}$, $F = \{q_1, q_4\}$, δ is given by following table

Input:	0			1			ϵ		
Stack:	0	$	ϵ	0	$	ϵ	0	$	ϵ
q_1									$\{(q_2, \$)\}$
q_2		$\{(q_2, 0)\}$	$\{(q_3, \epsilon)\}$						
q_3				$\{(q_3, \epsilon)\}$			$\{(q_4, \epsilon)\}$		
q_4									

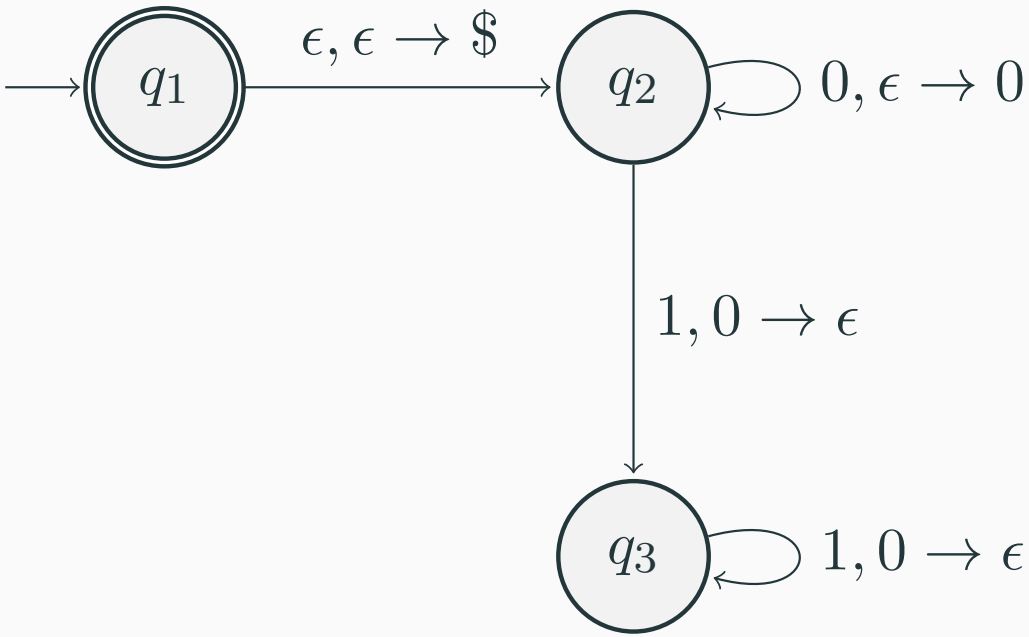
PDA for $\{0^n 1^n : n \geq 0\}$



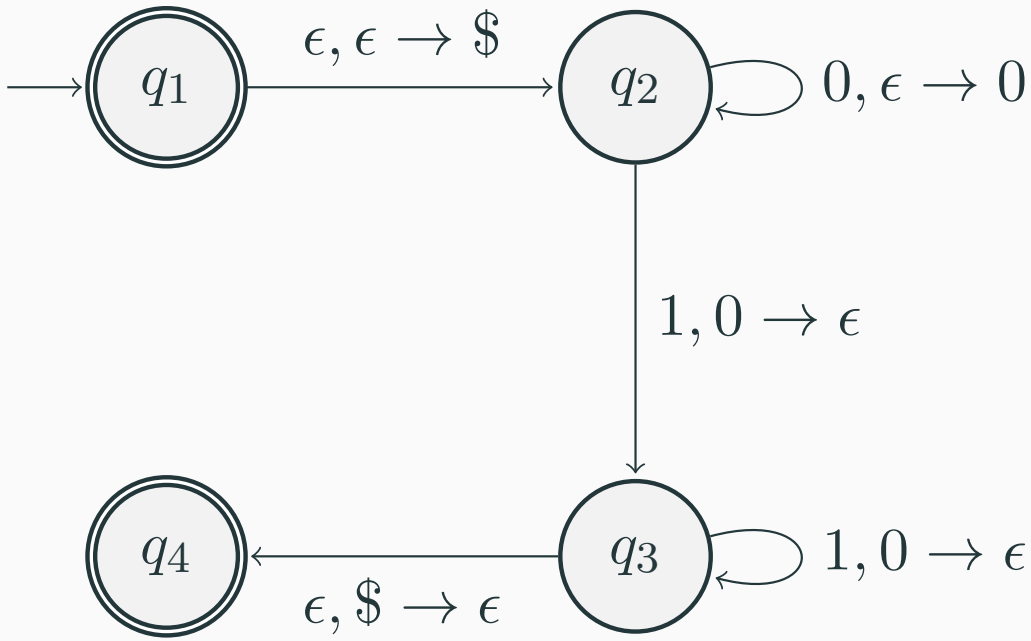
PDA for $\{0^n 1^n : n \geq 0\}$



PDA for $\{0^n 1^n : n \geq 0\}$



PDA for $\{0^n 1^n : n \geq 0\}$

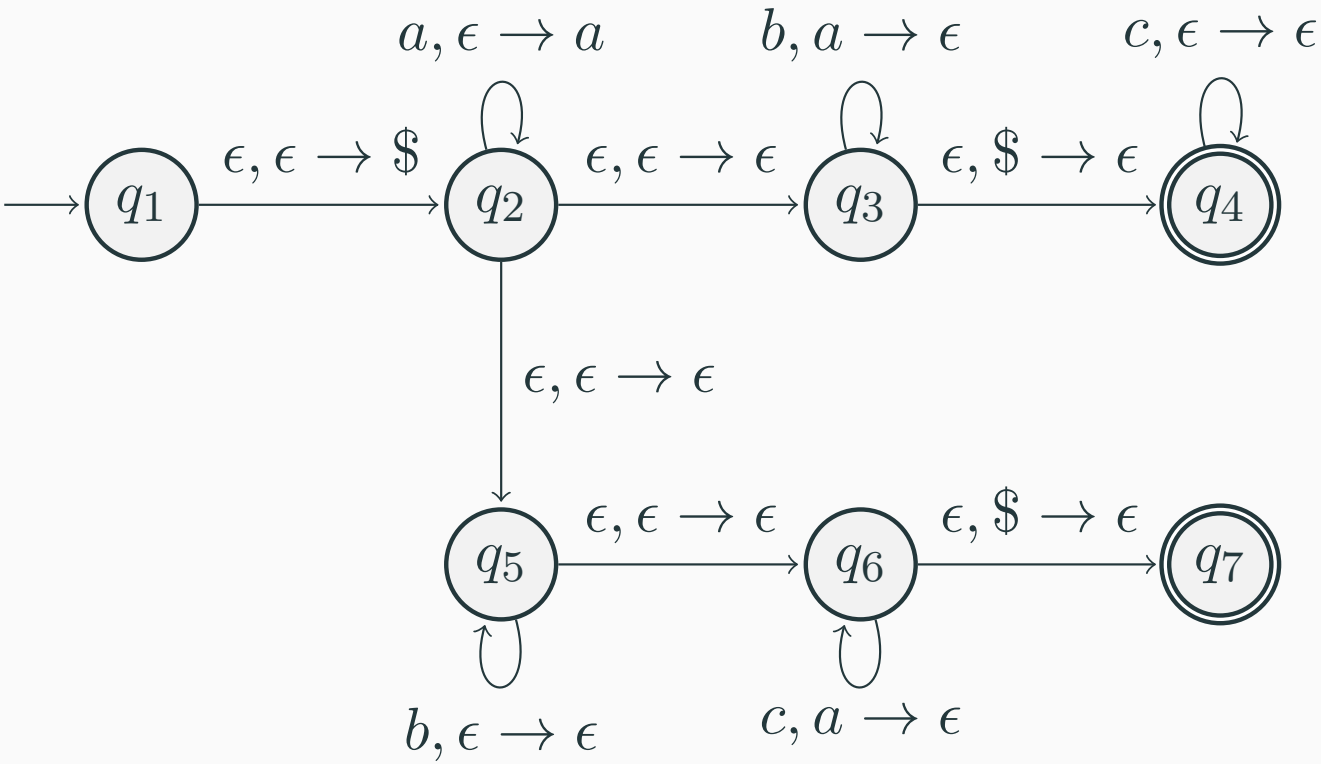


Example

$$\{a^i b^j c^k : i, j, k \geq 0, i = j \vee i = k\}$$

Example

$$\{a^i b^j c^k : i, j, k \geq 0, i = j \vee i = k\}$$

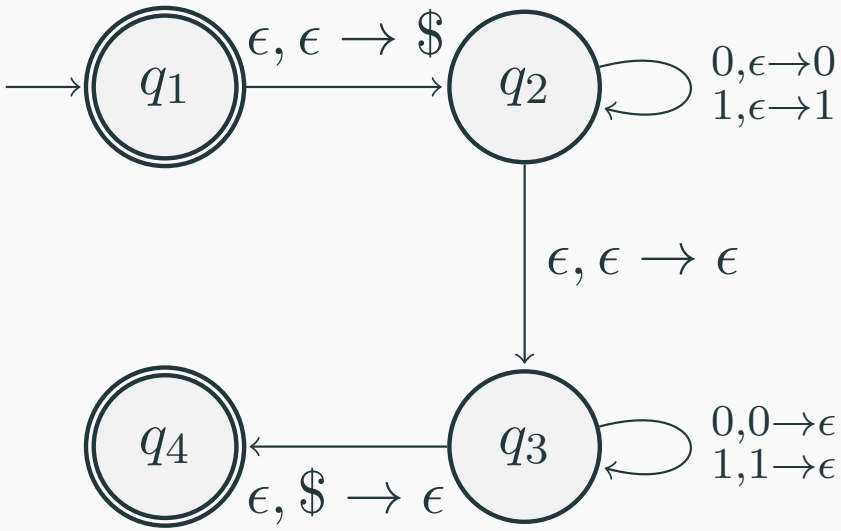


Example PDA

$$\{ww^{\text{rev}} : w \in \{0,1\}^*\}$$

Example PDA

$$\{ww^{\text{rev}} : w \in \{0,1\}^*\}$$



Equivalence of CFG and PDA

Theorem 2

A language is context free if and only if some pushdown automata recognizes it.

Equivalence of CFG and PDA

Theorem 2

A language is context free if and only if some pushdown automata recognizes it.

Corollary 3

Every regular language is context free.

Non-Context-Free Languages

Theorem 4 (Pumping Lemma for Context-Free Languages)

If A is a context-free language, then there is a number p (the pumping length) where, if s is a any string in A of length at least p, then s may be divided into five pieces $s = uvxyz$ satisfying the conditions

1. *for each $i \geq 0$, $uv^ixy^iz \in A$*
2. *$|vy| > 0$, and*
3. *$|vxy| \leq p$.*

Non-Context-Free Languages

- Let $B = \{a^n b^n c^n : n \geq 0\}$, B is not CFL
- Assume B is context-free
- Let p be the pumping length for B and let $s = a^p b^p c^p$, $s \in B$ and $|S| \geq p$
- Let $s = uvxyz$, either v for y is nonempty
- When both v and y contain only one type of symbols, v does not contain both a's and b's or both b's and c's (same holds for y) therefore uv^2xy^2z cannot contain equal number of a's, b's, and c's
- When either v or y contain more than one type of symbol uv^2xy^2z may contain equal numbers of the three alphabet symbols but not in correct order
- One of these two cases must occur which lead to contradiction

Non-Context-Free Languages

- Let $C = \{a^i b^j c^k : 0 \leq i \leq j \leq k\}$, C is not CFL

Non-Context-Free Languages

- Let $C = \{a^i b^j c^k : 0 \leq i \leq j \leq k\}$, C is not CFL
- C is similar to B
- Assume C is context-free

Non-Context-Free Languages

- Let $C = \{a^i b^j c^k : 0 \leq i \leq j \leq k\}$, C is not CFL
- C is similar to B
- Assume C is context-free
- Let p be the pumping length for B and let $s = a^p b^p c^p$, $s \in B$ and $|S| \geq p$

Non-Context-Free Languages

- Let $D = \{ww : w \in \{0,1\}^*\}$

Non-Context-Free Languages

- Let $D = \{ww : w \in \{0,1\}^*\}$
- D is not CFL
- Assume D is CFL and obtain a contradiction
- Let p be pumping length
- If $s = 0^p 1 0^p 1$ then we can pump it

Non-Context-Free Languages

- Let $D = \{ww : w \in \{0,1\}^*\}$
- D is not CFL
- Assume D is CFL and obtain a contradiction
- Let p be pumping length
- If $s = 0^p10^p1$ then we can pump it
- If we let $s = 0^p1^p0^p1^p$ then s cannot be pumped